

NAME: TANKISO MASOEBE

PROGRAMMING WITH C++ 2 WK9

1.Scenario Analysis: Exception Handling in an E-Commerce IT System

An e-commerce system must handle unexpected errors to ensure that customers can shop without losing data or payments. For example, a customer adds products to a shopping cart and proceeds to checkout. During payment processing, the bank server may become unavailable or the payment may be declined. Without proper exception handling, the system could crash or incorrectly confirm the order. The e-commerce system should use exception handling to detect the payment error and safely stop the transaction. It should display a clear message informing the customer that the payment could not be completed and allow them to try again. The system should also record the error in a log for administrators to investigate. If payment is successful but the order database temporarily fails, the system should prevent duplicate charges and preserve the transaction information for later processing. Effective exception handling improves system reliability, protects customer information, prevents financial errors, and provides a better overall shopping experience.

2. Concept Research: Standard Exceptions

Standard exceptions are predefined errors provided by programming languages to handle common problems during program execution. They help developers identify and manage unexpected situations without causing the entire program to crash. Examples include division by zero, invalid input, accessing an unavailable file, and trying to access an invalid array position. In languages such as C++, standard exceptions include `std::exception`, `std::runtime_error`, and `std::out_of_range`. Developers can use `try`, `catch`, and `throw` statements to detect and handle these exceptions. Standard exceptions make programs more reliable, easier to debug, and safer by providing clear ways to respond to errors and maintain normal program operation.

3. Tool Practice: Custom Exception

I created a custom exception in Visual Studio Code using C++. The program defines a `CustomException` class that inherits from `std::exception` and provides a specific error message. I used a `try` block to run the program and a `catch` block to handle the custom exception when an invalid situation occurs. This practice helped me understand how programmers can create their own error-handling mechanisms instead of relying only on standard exceptions. Custom exceptions make programs easier to understand, debug, and maintain because they describe specific problems clearly. Using Visual Studio Code also helped me practice writing, compiling, testing, and correcting exception-handling code efficiently.

4. Application Practice: Exception-Handling Function

An exception-handling function is designed to detect and manage errors that may occur while a program is running. For example, a C++ function can accept a user's age and throw an exception when an invalid value, such as a negative age, is entered. The function uses `try`, `throw`, and `catch` statements to handle the problem safely. Its purpose is to prevent the program from crashing and provide a meaningful message to the user. Exception-handling functions improve software reliability, simplify debugging, and allow programs to continue operating after recoverable errors. They are especially useful in systems that process user input, payments, files, or databases.